

GameReviewHub, la tua destinazione gaming

Francesca Avallone, Michele Letterese, Gabriel Balasa

Basi di dati 2

Contents

1	Introduzione	2
2	Descrizione del Miniworld	3
2.1	Fonte di dati	3
2.2	Entità principali	3
3	Contesto Applicativo	5
3.1	Utenti e casi d'uso	5
3.2	Estensibilità futura	6
4	Soluzione Proposta	7
4.1	Stack tecnologico utilizzato	7
4.2	Architettura utilizzata	7
4.3	Descrizione dettagliata delle funzionalità	8
4.3.1	Scenari d'uso	8
4.4	Conformità proprietà BASE e CAP	9
5	Metodologia utilizzata per sviluppare la soluzione	10
5.1	Implementazione del back-end	10
5.2	Implementazione del front-end	10
5.3	Strumenti e tecniche	11

1 Introduzione

Negli ultimi anni, la gestione e l'analisi dei dati nel settore dei videogiochi hanno acquisito un ruolo sempre più centrale, grazie alla crescente disponibilità di dataset pubblici che raccolgono informazioni dettagliate sui videogiochi, le recensioni degli utenti e le performance di vendita. Questo progetto nasce con l'obiettivo di realizzare una web application in grado di offrire un'interfaccia intuitiva per l'esplorazione e la consultazione di tali dati, sfruttando un approccio NoSQL attraverso l'utilizzo di MongoDB come sistema di gestione del database.

Il dataset utilizzato contiene informazioni eterogenee riguardanti i **videogiochi**, tra cui titolo, piattaforma, genere, anno di uscita, sviluppatore, publisher e classificazione.

A queste informazioni si affiancano dati relativi alle **recensioni**, che includono punteggio medio dei critici (critic score), numero di recensioni dei critici (critic count), punteggio medio degli utenti (user score) e numero di recensioni degli utenti (user count).

Inoltre, sono stati integrati dati sulle **vendite**, suddivise per area geografica (Nord America, Europa, Giappone, Altri Paesi) e il totale globale.

L'obiettivo principale del progetto è stato quello di progettare un database flessibile e scalabile, adatto alla natura semi-strutturata dei dati, e di sviluppare un'interfaccia web che consenta di effettuare ricerche, filtri e analisi interattive sui dati disponibili. L'adozione di MongoDB ha permesso di gestire in maniera efficiente la variabilità degli attributi e la complessità delle relazioni tra le entità coinvolte.

2 Descrizione del Miniworld

Il miniworld individuato per questo progetto riguarda il dominio dei videogiochi, con particolare attenzione agli aspetti legati alle informazioni descrittive dei titoli, alle recensioni da parte di utenti e critici, e ai dati sulle vendite globali. L'obiettivo è fornire una base dati che consenta analisi statistiche e consultazioni mirate, permettendo all'utente di esplorare le caratteristiche dei giochi e di confrontarne le performance di mercato e la ricezione critica.

2.1 Fonte di dati

I dati utilizzati sono stati estratti da un dataset pubblico disponibile sulla piattaforma Kaggle, al seguente indirizzo: <https://www.kaggle.com/datasets/thedevastator/video-game-sales-and-ratings/data>.

Il dataset contiene un'ampia raccolta di informazioni su numerosi videogiochi pubblicati negli ultimi decenni. Ogni record del dataset rappresenta un videogioco e include i seguenti gruppi di informazioni principali:

- **Caratteristiche del gioco:** titolo, piattaforma di rilascio, genere, anno di uscita, sviluppatore (*developer*), editore (*publisher*) e classificazione (*rating*).
- **Recensioni:** valutazioni provenienti sia da critici che da utenti, rappresentate da punteggi medi e conteggi. In particolare, sono presenti i campi *critic score*, *critic count*, *user score* e *user count*.
- **Vendite:** dati relativi alle copie vendute, suddivisi per area geografica (Nord America, Europa, Giappone, altri territori) e comprensivi del totale globale.

La scelta di questo dataset è stata motivata dalla sua ricchezza informativa e dalla possibilità di esplorare un dominio molto attuale e rilevante nel contesto dell'industria videoludica. La varietà dei dati e la loro natura semi-strutturata si prestano bene a un approccio NoSQL basato su documenti, come quello offerto da MongoDB.

2.2 Entità principali

Dall'analisi del dataset sono state individuate tre entità principali che costituiscono il nucleo informativo del nostro miniworld:

- **Videogioco:** rappresenta l'elemento centrale, con attributi come *nome*, *piattaforma*, *genere*, *anno di uscita*, *developer*, *publisher* e *rating*. Questa entità permette di identificare univocamente ogni gioco e fornisce le informazioni di base necessarie alla sua descrizione.

- **Review:** associata a ciascun videogioco, contiene le informazioni sulle recensioni e sul gradimento del titolo. Gli attributi principali sono:

- *critic score*: punteggio medio attribuito dai critici.
- *critic count*: numero totale di recensioni critiche considerate.
- *user score*: punteggio medio assegnato dagli utenti.
- *user count*: numero di utenti che hanno espresso una valutazione.

Questa entità consente analisi qualitative sui giochi, confrontando la percezione della critica con quella degli utenti.

- **Sales:** anch'essa collegata a ogni videogioco, rappresenta le vendite registrate a livello geografico. Gli attributi sono:

- *NA Sales*: vendite in Nord America.
- *EU Sales*: vendite in Europa.
- *JP Sales*: vendite in Giappone.
- *Other Sales*: vendite in altri territori.
- *Global Sales*: totale delle vendite globali.

Questa entità è fondamentale per comprendere l'andamento commerciale dei giochi e analizzare differenze tra mercati regionali.

La modellazione del miniworld è stata progettata per sfruttare le potenzialità di MongoDB, adottando una struttura a documenti annidati che permette di raggruppare le informazioni relative a recensioni e vendite all'interno del documento principale del videogioco. Questo approccio consente una gestione più naturale e flessibile dei dati, evitando relazioni complesse tipiche dei database relazionali.

```
_id: ObjectId('681b3bbdaaa2492a5200f7b')
id_game: 1002
name: "Mario Kart Wii"
platform: "Wii"
year_of_release: 2012
genre: "Racing"
publisher: "Nintendo"
developer: "Nintendo"
rating: "E"
```

Figure 1: Esempio: uno screen della collection 'games' con un documento aperto

3 Contesto Applicativo

Il contesto applicativo del progetto è rappresentato da una web application che consente di gestire un insieme di dati relativi ai videogiochi, alle loro recensioni e ai dati di vendita globali.

L'obiettivo principale è fornire un'interfaccia semplice e intuitiva che consenta all'utente di interagire con i dati in maniera completa, eseguendo operazioni di inserimento, modifica, eliminazione e ricerca.

L'applicazione è concepita come strumento libero, senza meccanismi di autenticazione, per facilitare l'accesso e la sperimentazione.

3.1 Utenti e casi d'uso

L'utenza prevista è costituita principalmente da un unico tipo di utente: l'amministratore del sistema. Tuttavia, l'applicazione è aperta e non richiede credenziali di accesso, permettendo a chiunque di utilizzarne le funzionalità.

I principali casi d'uso previsti sono i seguenti:

- **Inserimento di un nuovo videogioco:** consente di aggiungere un nuovo titolo al database, includendo contestualmente le informazioni sulle recensioni (critici e utenti) e sui dati di vendita. L'inserimento avviene in un'unica operazione, così da mantenere la coerenza dei dati.
- **Eliminazione di un videogioco:** permette di cancellare un gioco esistente dal database, rimuovendo automaticamente le informazioni sulle recensioni e sulle vendite associate.
- **Eliminazione selettiva:** l'utente può anche decidere di eliminare solo le informazioni relative alle recensioni oppure soltanto i dati sulle vendite, mantenendo il record principale del videogioco.
- **Modifica dei dati:** l'applicazione supporta l'aggiornamento delle informazioni, sia per i dati principali del gioco, sia per le sezioni di recensioni e vendite in modo indipendente.
- **Ricerca e filtraggio:** tramite la sezione dedicata alla ricerca, l'utente può esplorare il database applicando filtri specifici. In particolare, è possibile selezionare i giochi in base alla piattaforma e definire una soglia minima di punteggio (*user score* o *critic score*), simulando un'operazione di join tra le informazioni del gioco e le recensioni. Questa funzionalità è utile per individuare rapidamente titoli di interesse secondo criteri qualitativi.

Le operazioni CRUD implementate (Create, Read, Update, Delete) garantiscono una gestione completa e dinamica dei dati, sfruttando la flessibilità di MongoDB e l'interfaccia web per una fruizione semplice ed efficace.

3.2 Estensibilità futura

Il progetto è stato progettato tenendo conto della possibilità di espansioni future. Alcune direzioni di sviluppo potrebbero includere:

- **Gestione degli utenti e autenticazione:** introduzione di un sistema di registrazione e login per differenziare i ruoli (ad esempio amministratore e utente base), aumentando il livello di sicurezza.
- **Funzionalità di analisi avanzata:** implementazione di strumenti di reportistica e visualizzazione grafica per analizzare le tendenze di vendita, il confronto tra piattaforme o il rapporto tra valutazioni e performance commerciali.
- **Integrazione con API esterne:** possibilità di collegare l'applicazione con fonti dati esterne, come database di recensioni in tempo reale o marketplace digitali.
- **Arricchimento dei dati:** aggiunta di nuove informazioni per ciascun gioco, come immagini, trailer, descrizioni estese o collegamenti a store online.
- **Funzioni social:** inserimento di sezioni dedicate ai commenti o alla valutazione diretta da parte degli utenti, trasformando l'applicazione in una piattaforma interattiva.

La scelta di un database NoSQL come MongoDB si è rivelata strategica in questa ottica, poiché consente di aggiungere facilmente nuovi campi e strutture ai documenti senza modificare rigidamente lo schema esistente.

4 Soluzione Proposta

La soluzione sviluppata consiste in un sistema web per la gestione e visualizzazione di dati relativi ai videogiochi, alle recensioni e alle vendite globali. L'applicazione sfrutta un database NoSQL (MongoDB) per l'archiviazione dei dati, implementa le operazioni CRUD in maniera completa e offre un'interfaccia utente interattiva. Il progetto è stato realizzato seguendo il pattern architetturale Model-View-Controller (MVC) per garantire modularità e manutenibilità del codice.

4.1 Stack tecnologico utilizzato

Il progetto è stato realizzato utilizzando le seguenti tecnologie:

- **Database:** *MongoDB*, scelto per la sua natura document-oriented, che consente di gestire dati semi-strutturati in maniera flessibile e scalabile. La gestione è stata supportata da *MongoDB Compass* per la visualizzazione e il debugging dei dati.
- **Backend:** *Java Servlet*, implementando il pattern *Model-View-Controller (MVC)* per separare la logica applicativa dalla presentazione e garantire una migliore manutenzione del codice.
- **Frontend:** *JSP (Java Server Pages)* e framework *Bootstrap* per realizzare un'interfaccia utente semplice, responsiva e intuitiva.
- **IDE:** *IntelliJ IDEA*, utilizzato come ambiente di sviluppo integrato per scrivere, testare e debuggare il codice Java e le servlet.
- **Librerie e driver:** driver ufficiale MongoDB per Java, necessario per interagire con il database.

Questa combinazione garantisce un'integrazione semplice tra le componenti, mantenendo al contempo flessibilità nella gestione del modello dati NoSQL.

4.2 Architettura utilizzata

L'architettura adottata è di tipo *MVC*:

- **Model:** rappresenta le entità principali (videogioco, recensione, vendite) e si occupa dell'interazione diretta con MongoDB attraverso il driver Java.
- **View:** implementata tramite JSP e Bootstrap, gestisce la presentazione dei dati all'utente e offre moduli per le operazioni CRUD e di ricerca.
- **Controller:** implementato tramite Servlet Java, riceve le richieste dell'utente, invoca la logica del Model e aggiorna la View.

La comunicazione tra le componenti avviene in modo sincrono: il Controller processa le richieste HTTP (GET/POST), interroga MongoDB e invia i dati alle JSP per la visualizzazione.

4.3 Descrizione dettagliata delle funzionalità

L'applicazione implementa le principali operazioni di gestione dati:

- **Inserimento (Create):** aggiunta di un nuovo videogioco con le relative informazioni sulle recensioni e sulle vendite. L'inserimento avviene in un'unica operazione per mantenere la consistenza.
- **Visualizzazione (Read):** elenco dei videogiochi con possibilità di filtrare i risultati per piattaforma e punteggio minimo (operazione assimilabile a una JOIN tra attributi delle recensioni e i dati del videogioco).
- **Aggiornamento (Update):** modifica delle informazioni di un gioco, con possibilità di aggiornare singolarmente le sezioni *reviews* e *sales*.
- **Eliminazione (Delete):** rimozione completa di un videogioco (con le relative recensioni e vendite) oppure eliminazione selettiva di solo *reviews* o solo *sales*.

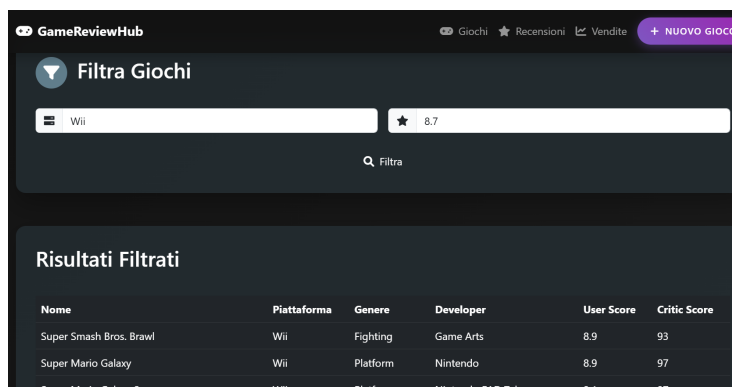


Figure 2: Ricerca tramite filtro sulla piattaforma e punteggio minimo

4.3.1 Scenari d'uso

Gli scenari d'uso principali sono:

1. **Amministratore inserisce un nuovo gioco:** tramite form dedicato, specificando tutte le informazioni necessarie (dati del gioco, recensioni e vendite).
2. **Amministratore cerca un gioco:** utilizzando filtri per piattaforma e punteggio minimo (critic score o user score).
3. **Amministratore modifica un gioco:** aggiornando le informazioni generali o i dati annidati relativi alle recensioni e vendite.

4. **Amministratore elimina un gioco:** con possibilità di cancellazione totale o parziale (solo review o sales).

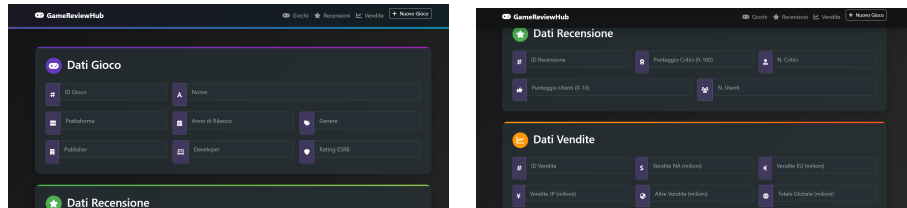


Figure 3: Esempio inserimento gioco specificando informazioni relative alle recensioni e vendite

4.4 Conformità proprietà BASE e CAP

Proprietà BASE Il progetto soddisfa le proprietà BASE tipiche dei sistemi NoSQL:

- **Basically Available:** il sistema è sempre disponibile per l'inserimento, la modifica e la lettura dei dati, salvo disconnessioni dal database.
- **Soft state:** non è previsto un vincolo rigido di consistenza immediata; i dati possono essere aggiornati in momenti diversi senza bloccare l'intero sistema.
- **Eventual consistency:** le modifiche apportate ai documenti diventano visibili a tutte le operazioni dopo un breve intervallo di tempo, coerentemente con il modello MongoDB.

Teorema CAP Il sistema rispetta due vincoli su tre, come tipico dei database distribuiti:

- **Availability (A):** le operazioni sono sempre disponibili agli utenti (o restituiscono un errore esplicito in caso di guasto).
- **Partition Tolerance (P):** MongoDB è in grado di funzionare in configurazioni distribuite, tollerando partizioni di rete.

Il requisito di **Consistenza (C)** è parzialmente sacrificato in favore di disponibilità e tolleranza alle partizioni, come previsto dal modello NoSQL.

5 Metodologia utilizzata per sviluppare la soluzione

Lo sviluppo della web application è stato organizzato seguendo un approccio incrementale, suddividendo le attività in tre macro-fasi: progettazione del database, implementazione del back-end e implementazione del front-end. Ogni fase è stata accompagnata da test continui per garantire la correttezza delle operazioni CRUD e della visualizzazione.

5.1 Implementazione del back-end

Il back-end è stato sviluppato in linguaggio **Java**, utilizzando le **Servlet** per la gestione delle richieste HTTP. L'architettura segue il pattern **Model-View-Controller (MVC)**:

- **Controller (Servlet)**: riceve le richieste dell'utente (GET e POST), valida i dati, richiama le operazioni sul Model e inoltra le risposte alla View.
- **Model**: contiene la logica di accesso ai dati e implementa le operazioni CRUD sul database MongoDB.

Le principali funzionalità implementate nel back-end includono:

- Connessione a MongoDB tramite **driver ufficiale Java**.
- Inserimento di un nuovo videogioco e delle relative informazioni (*reviews* e *sales*) in un'unica operazione.
- Cancellazione completa o selettiva (solo *reviews* o solo *sales*).
- Aggiornamento dei documenti, con gestione dei campi annidati.
- Funzione di ricerca con filtri, che simula una *join* tra attributi delle diverse sezioni del documento (es. filtraggio per piattaforma e punteggio minimo).

5.2 Implementazione del front-end

Il front-end è stato sviluppato utilizzando **JSP (Java Server Pages)** per la generazione dinamica delle pagine e **Bootstrap** per garantire un'interfaccia responsive e user-friendly.

Sono state implementate le seguenti interfacce:

- **Homepage**: consente la navigazione verso le sezioni principali (inserimento, ricerca, gestione giochi).
- **Form di inserimento**: permette di aggiungere un nuovo videogioco con dati relativi a recensioni e vendite.
- **Ricerca con filtri**: permette di cercare i giochi per piattaforma e punteggio minimo.

- **Sezione di modifica ed eliminazione:** permette l'aggiornamento delle informazioni o la cancellazione (parziale o totale) di un gioco.

La comunicazione tra front-end e back-end avviene tramite richieste **HTTP GET e POST**, gestite dalle Servlet. I dati inseriti nei form sono validati lato server prima di essere inviati al database.

5.3 Strumenti e tecniche

Per lo sviluppo e il testing del progetto sono stati utilizzati i seguenti strumenti:

- **IDE:** IntelliJ IDEA, utilizzato per scrivere, organizzare e debuggare il codice Java.
- **Database:** MongoDB, con **MongoDB Compass** per la gestione visuale delle collection e il controllo dei dati.
- **Controllo versioni:** Git (opzionale, se lo hai usato).
- **Librerie:** driver ufficiale MongoDB per Java, necessario per la connessione al database.

La tecnica principale seguita nello sviluppo è stata la **prototipazione incrementale**: inizialmente sono state implementate le operazioni **CRUD** di base, successivamente le funzionalità avanzate come la ricerca con filtri e la gestione delle operazioni parziali (solo *reviews* o *sales*).